

0–k BFS: Bounded Integer Weights Shortest Path Problem

Project Plan – Algorithms Course, C++ Implementation

Artemov Makar, Mark Shkut
Skoltech, Algorithms 2026

May 13, 2026

1. Motivation and Problem Statement

The classical single-source shortest path (SSSP) problem on a weighted graph $G = (V, E, w)$ with non-negative weights is solved by Dijkstra’s algorithm in $O((V + E) \log V)$ with a binary heap, or $O(E + V \log V)$ with a Fibonacci heap. However, when edge weights are restricted to a small set of non-negative integers $\{0, 1, \dots, k\}$ – a setting that arises frequently in grid path planning, layered graphs, 2-SAT-like reductions, transport networks with a few toll levels and many competitive-programming problems – one can replace the priority queue with a structure of $O(k)$ FIFO queues and obtain a strictly faster $O(kV + E)$ algorithm. The aim of this project is to implement, analyse and empirically compare the family of *0–k BFS* algorithms against Dijkstra-style baselines in C++.

2. Objectives

- Implement a reusable C++17 graph library with three SSSP backends: (i) 0–1 BFS using `std::deque`; (ii) 0–k BFS via edge subdivision; (iii) 0–k BFS via multiple FIFO queues with the circular $(k+1)$ -queue optimisation.
- Implement two Dijkstra baselines: `std::priority_queue` (binary heap) and an indexed d -ary heap.
- Prove correctness and derive the running time / memory of each variant; identify exactly when 0–k BFS beats Dijkstra.
- Benchmark on synthetic and real graph families and report results.

3. Algorithms to Be Implemented

(a) 0–1 BFS. Edges have weights in $\{0, 1\}$. Replace the BFS queue with a deque: relax an edge (v, u) of weight 0 by `push_front`, of weight 1 by `push_back`. The inf-check of plain BFS is replaced by the relaxation $dist[v] + w(v, u) < dist[u]$, which is required because a vertex may be reached several times before its final distance is fixed. Complexity $O(V + E)$, memory $O(V)$.

(b) 0–k BFS via edge subdivision. Each edge of weight $w > 1$ is split into w unit edges through $w - 1$ fresh dummy vertices, reducing the problem to 0–1 BFS on a graph with $V' \leq (k - 1)E + V$ vertices and $E' \leq kE$ edges. Total complexity $O(kE + V)$, memory $O(kE + V)$.

(c) 0–k BFS via multiple queues. Maintain queues $Q_0, Q_1, \dots$ where Q_i holds vertices currently at $dist = i$, and process them in non-decreasing order. A naive bound needs $k(V - 1)$ queues; since at any moment at most $k + 1$ queues are non-empty (a relaxation moves a vertex by at most k steps forward), it suffices to keep a circular buffer of $k + 1$ queues indexed by $dist[v] \bmod (k + 1)$. Complexity $O(kV + E)$, memory $O(k + E)$.

(d) **Dijkstra baselines** with `std::priority_queue` and a hand-rolled 4-ary indexed heap supporting `decrease_key`.

4. Theoretical Comparison

Algorithm	Time	Memory
BFS (unit weights)	$O(V + E)$	$O(V)$
0–1 BFS (deque)	$O(V + E)$	$O(V)$
0–k BFS, subdivision	$O(kE + V)$	$O(kE + V)$
0–k BFS, $k+1$ circular queues	$O(kV + E)$	$O(k + E)$
Dijkstra, binary heap	$O((V + E) \log V)$	$O(V + E)$
Dijkstra, Fibonacci heap	$O(E + V \log V)$	$O(V + E)$

For fixed k and $E = \Theta(V)$ both 0–k variants are asymptotically $\log V$ faster than a heap-based Dijkstra; the gap shrinks once $k = \omega(\log V)$.

5. Experimental Plan

- **Graph families:** (i) random $G(n, p)$ with weights drawn from $\{0, \dots, k\}$; (ii) 4-connected grids with terrain costs (a classic 0–k case); (iii) layered DAGs; (iv) DIMACS USA road network with weights quantised to $k \in \{4, 8, 16\}$; (v) adversarial chain graphs that force many relaxations.
- **Scale target.** Following the reviewer’s note we aim for V as close to 10^9 as the workstation allows. Explicit-edge backends top out around $V \approx 10^8$ (a CSR with $E = 4V$ and `uint32_t` indices already costs ≈ 20 GB); *implicit grid graphs* make $V = 10^9$ feasible because the neighbour function is computed in $O(1)$ and only the distance array (≈ 4 GB at `uint32_t`) lives in memory.
- **Parameters:** $V \in \{10^6, 10^7, 10^8, 10^9\}$ for grids, $V \in \{10^6, 10^7\}$ for general graphs; $k \in \{1, 2, 4, 8, 16, 64\}$; edge density sparse / dense.
- **Metrics:** wall-clock time (Google Benchmark), `perf`-counted instructions and L1/LLC misses, peak RSS, number of relaxations and queue operations.
- **Validation:** every backend’s output is checked against the `std::priority_queue` Dijkstra on every test instance.

6. Expected Limitations

0–k BFS requires (a) non-negative integer weights with a small known bound k , (b) a graph small enough that $O(kE)$ subdivision memory fits, or the circular-queue trick is used. It does not generalise to real-valued weights, to negative edges (Bellman–Ford / SPFA territory), or to problems requiring the full SSSP shortest-path DAG with tie-breaking by hop count. For large k ($k \gtrsim \log V$) heap-based Dijkstra is the better choice.

References

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein. *Introduction to Algorithms*, 4th ed., MIT Press, 2022.
- [2] E. W. Dijkstra. “A note on two problems in connexion with graphs.” *Numer. Math.* 1 (1959), 269–271.
- [3] R. K. Ahuja, K. Mehlhorn, J. B. Orlin, R. E. Tarjan. “Faster algorithms for the shortest path problem.” *J. ACM* 37(2), 1990. [4] CP-Algorithms team. “0–1 BFS,” cp-algorithms.com/graph/01_bfs.html.